

# EXPERIMENT 6

## Title

Create a stored procedure `transfer_employee(emp_id, new_dept_id)` with validation and error handling. Implement triggers for salary validation and audit logging. Test the procedure and triggers using normal and edge-case inputs.

## Aim

To create and execute a stored procedure for transferring an employee between departments with proper validation and error handling, and to implement database triggers for salary validation and employee salary audit logging.

## Theory

A stored procedure is a named group of SQL statements stored inside the database and executed using a CALL statement. Stored procedures can accept parameters, perform validation, modify data, and handle errors. They are useful for implementing reusable business logic close to the database.

- Procedure parameters allow values such as employee ID and new department ID to be supplied at execution time.
- Validation checks whether the employee exists, whether the target department exists, and whether the employee is already in that department.
- Transactions allow multiple related changes to be treated as one unit. COMMIT permanently saves successful changes, while ROLLBACK reverses changes when an error occurs.
- An exception handler can capture SQL errors and roll back a transaction.
- A BEFORE UPDATE trigger can validate a new salary before it is stored.
- An AFTER UPDATE trigger can record salary changes in an audit table.
- Triggers execute automatically when the associated INSERT, UPDATE, or DELETE event occurs.
- Edge-case testing checks invalid employee IDs, invalid departments, duplicate transfers, invalid salaries, and valid boundary values.

## Database Schema Used

```
CREATE TABLE Department (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(50) UNIQUE NOT NULL,  
    location VARCHAR(50)  
);
```

```
CREATE TABLE Employee (  
    employee_id INT PRIMARY KEY,  
    employee_name VARCHAR(60) NOT NULL,  
    job_title VARCHAR(50),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    department_id INT,  
    manager_id INT NULL,  
    FOREIGN KEY (department_id)
```

```
REFERENCES Department(department_id)
);
```

## Create Audit Table

```
CREATE TABLE salary_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT NOT NULL,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    changed_by VARCHAR(100)
);
```

## Stored Procedure: transfer\_employee

The procedure transfers an employee from the current department to a new department. It validates the input and uses a transaction so that the transfer is completed safely.

### Procedure Requirements

- The employee must exist.
- The target department must exist.
- The target department must be different from the employee's current department.
- The transaction should be rolled back if an SQL exception occurs.
- A success message should be returned after a successful transfer.

### Create the Procedure

```
DELIMITER $$

CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_current_dept INT;
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Validate employee
    SELECT COUNT(*)
    INTO v_emp_count
    FROM Employee
    WHERE employee_id = p_emp_id;

    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Employee does not exist';
```

```

END IF;

-- Validate department
SELECT COUNT(*)
INTO v_dept_count
FROM Department
WHERE department_id = p_new_dept_id;

IF v_dept_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Target department does not exist';
END IF;

-- Get current department
SELECT department_id
INTO v_current_dept
FROM Employee
WHERE employee_id = p_emp_id;

-- Prevent duplicate transfer
IF v_current_dept = p_new_dept_id THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Employee is already in this department';
END IF;

-- Perform transfer
UPDATE Employee
SET department_id = p_new_dept_id
WHERE employee_id = p_emp_id;

COMMIT;

SELECT 'Employee transferred successfully' AS message;
END$$

DELIMITER ;

```

## Execute the Stored Procedure

### View Employee Before Transfer

```

SELECT employee_id,
       employee_name,
       department_id
FROM Employee
WHERE employee_id = 1001;

```

### Transfer Employee

```
CALL transfer_employee(1001, 2);
```

### Verify Transfer

```

SELECT employee_id,
       employee_name,
       department_id
FROM Employee
WHERE employee_id = 1001;

```

## Salary Validation Trigger

```
DELIMITER $$

CREATE TRIGGER validate_employee_salary
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot be below 15000';
    END IF;

    IF NEW.salary > 1000000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot exceed 1000000';
    END IF;
END$$

DELIMITER ;
```

## Salary Audit Logging Trigger

```
DELIMITER $$

CREATE TRIGGER audit_employee_salary
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO salary_audit (
            employee_id,
            old_salary,
            new_salary,
            changed_by
        )
        VALUES (
            NEW.employee_id,
            OLD.salary,
            NEW.salary,
            CURRENT_USER()
        );
    END IF;
END$$

DELIMITER ;
```

## Test Salary Validation

### Valid Salary Update

```
UPDATE Employee
SET salary = 90000
WHERE employee_id = 1001;
```

### Salary Below Minimum

```
UPDATE Employee
SET salary = 10000
WHERE employee_id = 1001;
```

Expected result: The trigger rejects the update and displays the validation error.

### **Salary Above Maximum**

```
UPDATE Employee
SET salary = 2000000
WHERE employee_id = 1001;
```

Expected result: The trigger rejects the update because the salary exceeds the maximum allowed value.

### **Boundary Values**

```
UPDATE Employee
SET salary = 15000
WHERE employee_id = 1001;
```

```
UPDATE Employee
SET salary = 1000000
WHERE employee_id = 1001;
```

Expected result: Both boundary values are accepted because the rule uses greater-than-or-equal and less-than-or-equal limits.

### **Test Audit Logging**

```
SELECT audit_id,
       employee_id,
       old_salary,
       new_salary,
       changed_at,
       changed_by
FROM salary_audit
ORDER BY audit_id DESC;
```

### **Test Transfer Procedure Edge Cases**

#### **Invalid Employee ID**

```
CALL transfer_employee(9999, 2);
```

Expected result: Error – Employee does not exist.

#### **Invalid Department ID**

```
CALL transfer_employee(1001, 999);
```

Expected result: Error – Target department does not exist.

#### **Transfer to Same Department.**

```
SELECT employee_id, employee_name, department_id
FROM Employee
WHERE employee_id = 1001;
```

```
CALL transfer_employee(1001, 2);
```

Expected result: If employee 1001 is already in department 2, the procedure rejects the transfer.

## Valid Transfer

```
CALL transfer_employee(1001, 3);
```

Expected result: Employee 1001 is transferred to department 3, provided department 3 exists and is different from the current department.

## Verify Complete Employee Record

```
SELECT e.employee_id,  
       e.employee_name,  
       e.job_title,  
       e.salary,  
       e.department_id,  
       d.department_name  
FROM Employee e  
JOIN Department d  
  ON e.department_id = d.department_id  
WHERE e.employee_id = 1001;
```

## Verify Audit History for an Employee

```
SELECT *  
FROM salary_audit  
WHERE employee_id = 1001  
ORDER BY changed_at DESC;
```

## Result

The transfer\_employee stored procedure was created successfully with input validation, transaction handling, and error handling. Salary validation and audit logging triggers were implemented successfully, and normal as well as edge-case scenarios were tested.

## Conclusion

Stored procedures and triggers provide a powerful way to implement business rules directly inside a database. The transfer\_employee procedure centralizes employee transfer logic and protects the database from invalid input. The salary validation trigger prevents invalid salary values, while the audit trigger automatically records salary changes. Together, these features improve data integrity, consistency, and traceability.